Operation Systems Project
Electrical Computer and Engineering Dept.
Aristotle University of Thessaloniki
November 2025

TinyShell

The TinyShell is a small, Unix shell written in C for Linux, designed to mirror the basic behavior of a modern command interpreter while remaining lightweight and educational. The project evolves through multiple phases, each adding new system-level capabilities based directly on POSIX system calls.We are currently focused on completing the core requirements of Phase 1 and 2 of the TinyShell project.The Phase 1 and 2 establish the fundamental structure necessary for any command-line interpreter and validates the core process control mechanisms upon which all future capabilities will be built.In the Phase 3, the TinyShell has been evolved to support Job Control, a set of features allowing the user to manage multiple process groups (jobs) within a single terminal session. This phase transitions the shell to an asynchronous process manager, utilizing POSIX signals and terminal control functions. It consists of the core command loop, achieving the following functionalities:

1. Displays a prompt,
2. Parses the arguments,
3. Locates executables within the PATH,
4. Executes programs using `fork` and `execve`,
5. Overwrite files using the > operator and to append to files using >>
6. Support pipelines via the pipe()
7. Reports the exit codes, and
8. Can run external programs such as `ls`, `cat`, or `echo`, `grep`, terminating gracefully either with EOF or the `exit` command.
9. Manages data structures
10. Handles signals and asynchronous updates
11. Executes Foreground and Background
12. Controls jobs and built-in commands

The detailed breakdown of the implemented features is described in the following sections.

Headers and Preproccessor Directives

This initial section defines the project's dependencies and crucial constants, ensuring the shell has access to necessary system calls and external libraries.

- **POSIX/System Headers:**

- `#include <stdio.h>`, `#include <stdlib.h>`, `#include <string.h>`: Standard C library headers for basic input/output, general utilities, and string manipulation.
- `#include <unistd.h>`: Essential for POSIX calls such as `fork()` and `access()` (used in Path Resolution).
- `#include <sys/wait.h>`, `#include <sys/types.h>`: Vital headers for process management, specifically providing the `waitpid()` function and macros (`WIFEXITED`, `WEXITSTATUS`) required for reporting the child process status. The need for these headers necessitates the installation of the `build-essential` package on Linux environments.
- `#include <fcntl.h>`: This header provides the essential symbolic constants (bitmasks) that dictate how a file is accessed and modified. The specific flags utilized in the `handleRedirection` function include: `O_CREAT`, `O_WRONLY` / `O_RDONLY`, etc.
- `#include <signal.h>`: It is critical for managing how the shell and its child processes (SIGCHLD management) respond to external interrupts and internal state changes.
- `#include <termios.h>`: Provides terminal interface control, thus the API for controlling the terminal's hardware-level settings, such as input/output modes, manages sessions and process group ownership of the TTY.
- `#include <errno.h>`: Reports system errors, by defining a global variable errno.It identificates errors when a system call such as fork() fails. Utilizes the perror() function, which translates the current errno value into a human-readable string and prints it to stderr. This is essential for debugging issues.

- **External Library Headers (Readline):**
  - `#include <readline/readline.h>`, `#include <readline/history.h>`: These include files integrate the GNU Readline library, which provides advanced terminal editing features (e.g., backspace, cursor movement) and command history management. Their usage mandates linking the library during compilation via the `-lreadline` flag, and installing the `libreadline-dev` package.

- **Preprocessor Directives (Constants):**
  - `#define MAX_ARG_CAPACITY 50`: Defines the initial capacity for the dynamically allocated arguments array, controlling the initial memory footprint and reducing the frequency of `realloc()` calls.
  - `#define ARG_DELIMITERS " \t\r\n\a"`: Defines the set of characters (`space`, `tab`, `carriage return`, `newline`, `alert`) used by `strtok()` to effectively tokenize the user's input string, ensuring robustness against varying whitespace usage.
  - `#define MAX_JOBS 20`: It is the upper limit of simultaneous jobs and concurrent processes the shell can monitor at any given time.

Core Command Loop and User Input Handling

The foundation of the TinyShell is the Core Command Loop, which is implemented inside the shellMainLoop() function, that governs the entire lifecycle of the command interpreter. This loop runs continuously, maintaining the shell's state until explicitly terminated.The implementation of input handling transitions from basic C library functions (like getline) to the more

robust GNU Readline library to enable advanced shell features and prevent potential read errors. The relevant code is split between the loop function, shellMainLoop(), and the dedicated input function, getInputLine(). The critical task of gathering user input is managed by the function getInputLine(), which leverages the GNU Readline library. This integration is essential as it moves beyond basic buffered I/O to provide robust line editing capabilities, including managing the backspace key and resolving input buffer errors that plague simpler implementations. Furthermore, Readline automatically displays the shell's prompt (TinyShell>) and manages command history, storing previous commands for retrieval. After reading, the loop checks for the EOF (Ctrl+D) condition, which, along with the built-in exit command, ensures the shell can terminate correctly and return control to the host operating system.To accommodate the requirements of Phase 2, the loop's architecture was significantly expanded. Following input acquisition, a pre-emptive scanning mechanism was introduced to detect the pipe delimiter (`|`). If detected, the control flow immediately diverges to the dedicated executePipedCommands function, bypassing standard parsing to coordinate multi-stage Inter-Process Communication (IPC). Conversely, for standard commands, the execution logic was enhanced to support I/O Redirection. The handleRedirection function is now invoked specifically within the child process creation block, manipulating file descriptors to route streams (<, >, >>) prior to the overlaying of the program image via execve. Finally, the Phase 3 includes complex Job Control logic, allowing the shell to manage multiple processes and their interaction with the terminal. We will represent the step-by-step analysis of the shellMainLoop() code, focusing on the system-level mechanics and POSIX principles involved in each step.

- **Loop Initialization and Argument Acquisition:** The loop starts and attempts to read the user's command line.It initiates by `while (isRunning)`: The main loop keeps the shell active until the isRunning flag is explicitly set to 0 (or exit is called).Then loop continues by acquiring user input through inputLine = getInputLine(), which relies on the Readline library. This step is crucial as Readline handles sophisticated terminal I/O (like backspace and history), delivering a clean, fully processed string. This string is then immediately passed to commandArgs = splitInputIntoArguments(inputLine), which tokenizes the command line using delimiters to produce the null-terminated array of strings (char **) required by POSIX execution functions. Immediate checks follow to handle EOF (Ctrl+D) and empty lines, ensuring the shell terminates or ignores non-commands, respectively.

- **Command Identification and Built-ins:** The shell prioritizes checks for internal commands to avoid unnecessary process creation. The built-in `exit` command is handled by retrieving an optional exit code, performing essential **memory cleanup** (`free()` operations), and forcing process termination via `exit(exitCode)`.

- **Change Directory (cd):** The implementation of the `cd` command is fundamentally a demonstration of the core distinction between **built-in commands** and **external programs** in a POSIX shell. Unlike programs such as `ls` or `cat` (which are executed via `fork` and `execve` process model), `cd` must execute **internally** within the shell's own process. This necessity stems from the fact that its goal is to modify the shell's **Current Working Directory (CWD)**. Executing `cd` in a child process would only change the directory for that child, which immediately terminates, leaving the parent shell unaffected. Therefore, the implementation utilizes the direct POSIX system call `chdir(path)`, which modifies the CWD of the active process. The shell first manages logical handling, such as resolving the user's home directory (`getenv("HOME")`) when no path is provided. If `chdir` is successful, the execution flow is **diverted** by using the `continue` statement, which bypasses

the standard `fork/execve` sequence and returns immediately to the prompt. This ensures that the shell's environment is updated instantly and the shell is ready to execute the next command from the new location.

- **Path Resolution and Validation:** For external commands, the shell calls `execPath = resolveCommandPath(...)`. This function encapsulates the **Path Resolution** logic, iterating through directories specified in the `PATH` environment variable and using the POSIX `access(path, X_OK)` call to verify the executable's existence and permissions before execution.

- **POSIX Process Management (`fork`, `execve`, `waitpid`):** This is the central process orchestration block, triggered only when an executable path is successfully resolved.The `fork()` call duplicates the shell process, creating an identical **child process** (`childPID == 0`).The child's sole purpose is to execute the target program. execve(execPath, commandArgs, environ) overlays the child's process space with the new program's code. If this fails (e.g., file permissions changed), the child must immediately call `exit(EXITFAILURE)` to avoid continuing the shell's logic loop. Simultaneously, the parent process enters a blocking `do-while` loop centered on `waitpid()`, guaranteeing the shell's operation remains synchronized with the external program's lifecycle .The shell remains responsive only after the program has completed its task (either by exiting normally or being terminated by a signal) and waits until the child process has terminated.

- **Pipeline Detection and Priority Branching:** A critical control flow mechanism was introduced immediately after input acquisition.
  - **The Logic:** The code uses `strchr(inputLine, '|')` to scan the raw input string for the pipe character.
  - **Priority Execution:** If a pipe is detected, the shell immediately diverges from the standard parsing logic. It calls `executePipedCommands(inputLine)` to handle the multi-stage execution and then executes a `continue` statement.
  - **Rationale:** This "early exit" strategy is necessary because the standard tokenizer (`splitInputIntoArguments`) splits input based on whitespace. It is ill-equipped to handle the complex structure of a pipeline string. Therefore, piped commands are offloaded to their own dedicated parser before standard processing occurs.

- **Integration of Redirection in Standard Execution:** For standard commands (those without pipes), the execution block was modified to support I/O redirection (e.g., `ls > out.txt`).
  - **Injection Point:** The call to `handleRedirection(commandArgs)` was inserted specifically within the **Child Process block** (`if (childPID == 0)`), immediately *after* `fork()` but strictly *before* `execve()`.
  - **Process Isolation:** Placing this call inside the child process is architecturally critical. Because `handleRedirection` modifies the process's file descriptor table (swapping `STDOUT/STDIN`), executing it in the parent would permanently alter the shell's own I/O streams, causing it to stop printing to the terminal. By placing it in the child, the redirection affects only the transient process running the command.

- **Background Detection:** The TinyShell pre-emptively scans the input for the & symbol at the end of the line. If present, it flags the command as a background job (is_bg = 1) and removes the symbol before parsing. Unlike standard execution, which enters a

blocking waitpid() loop, background jobs allow the parent shell to execute a continue or simply reach the end of the loop, returning immediately to the TinyShell> prompt. Also, background jobs can't own the terminal, because they are specifically excluded from the tcsetpgrp(shell_terminal, pid) call. Because the shell does not wait for background jobs, it relies on the sigchld_handler to catch the SIGCHLD signal when the job eventually terminates. The handler then calls delete_job(pid) to clean up the entry in the MAX_JOBS table.

- **Foreground Management:** For non-background jobs, the loop uses tcsetpgrp() to grant the child process control of the terminal and enters a blocking waitpid() state. Once the foreground job terminates or is stopped, the shell must regain control. It calls tcsetpgrp(shell_terminal, shell_pgid), effectively restoring the shell's ability to display the TinyShell¿ prompt and receive new commands. The built-in command **fg** provided by the shell moves a previously backgrounded or stopped job into the foreground. It sends a SIGCONT signal to the entire process group to wake it up and immediately calls tcsetpgrp() to give that group terminal ownership and then enters the standard waitpid() blocking loop, just as if the command had been started in the foreground initially. When the user presses Ctrl+Z while a process is running in the foreground, the TinyShell manages a complex transition to safely suspend the process group and return control to the shell prompt. Then the loop executes its final cleanup and returns to the top, where getInputLine() displays the prompt, ready for the next command.

- **Signal Isolation and Process Group:** Before spawning any process, the loop blocks SIGCHLD signals using sigprocmask to prevent race conditions during job table updates. Inside the fork() block, setpgid(0, 0) is called to ensure the new process becomes a leader of its own group, isolating it from shell-level signals like Ctrl+C.

- **Built-in Priority:** The shell prioritizes internal checks primarily to modify its own environment or manage its internal state. Commands like cd must execute within the shell's process to modify the Current Working Directory (CWD) of the parent shell itself. By identifying internal commands via strcmp, the shell avoids the performance cost of fork() and execve() for operations that do not require external binaries.

- **Exit Status Reporting and Cleanup:** Upon termination of the child process, the shell fulfills the crucial requirement of reporting the outcome. The status is checked using `WIFEXITED` (or `WIFSIGNALED` for signals), and the actual exit code is extracted via `WEXITSTATUS(childStatus)` and reported via `fprintf(stderr, "[Exit Status: %d]", ...)`. Finally, meticulous cleanup is performed using `free(inputLine)` (for Readline's allocation), `free(commandArgs)`, and `free(execPath)` to prevent memory leaks and maintain resource integrity.

Readline Integration: Advanced I/O with GNU Readline Library

- **Prompt Display and Input Reading:** The core operation is executed by `char *line = readline("TinyShell> ");`. The `readline()` function is highly efficient: it simultaneously displays the specified prompt string (`TinyShell> `) and manages robust terminal input. This single call automatically handles line editing (including backspace functionality), preventing the input buffering issues common with simpler `getline()` implementations.

- **EOF Handling:** The conditional check `if (line == NULL)` handles the End-Of-File (EOF) condition, typically triggered by the user pressing `Ctrl+D`. If `readline()` returns `NULL`, the shell interprets this as a signal for graceful termination and returns `NULL` to the main loop.
- **History Management:** The block `if (*line) { add_history(line); }` ensures that every non-empty command entered by the user is added to the shell's history stack. The `add_history()` function then allows the user to navigate and reuse previous commands via the Up and Down arrow keys.
- **Memory Allocation:** It is important to note that `readline()` dynamically allocates memory for the input string. This mandates that the calling function, `shellMainLoop()`, must include a corresponding `free(inputLine)` call after the command has been processed, ensuring memory integrity and preventing leaks across continuous loop iterations.
- **Function Return Value Analysis** The `getInputLine()` function is designed to return a value that dictates the next action of the `shellMainLoop()`:
  - **Successful Return (Non-NULL Pointer):** The function returns a valid `char*` pointer containing the user's command line string (including non-empty strings and strings consisting only of spaces). This indicates that a command was successfully read and should be processed by the shell.
  - **Termination Signal (NULL Pointer):** The function returns `NULL` under the specific condition where the user inputs the **End-Of-File (EOF)** signal (typically `Ctrl+D`). This return value is immediately checked in the main loop to set the `isRunning` flag to false, ensuring the shell terminates gracefully.

Argument Parsing

The `splitInputIntoArguments()` function is responsible for the critical task of transforming the raw user input line into a structured format suitable for the `execve()` system call. This process involves tokenization and dynamic memory management.

- **Initial Allocation and Error Handling:** The function begins by dynamically allocating memory for the array of argument pointers (`char** arguments`). It uses a predefined initial capacity (`MAX_ARG_CAPACITY`) and includes a critical check (`if (!arguments)`) to handle potential memory allocation failures, terminating the shell gracefully if the OS cannot provide the necessary resources.
- **Tokenization with `strtok()`:** The function leverages the C standard library function `strtok()` for tokenization.
  - The initial call (`currentArg = strtok(inputLine, ARG_DELIMITERS)`) breaks the input string into the first token (the command name), automatically **ignoring any leading delimiters** (whitespace, tabs, etc.).
  - Subsequent calls (`currentArg = strtok(NULL, ARG_DELIMITERS)`) continue the process, returning the remaining arguments until no more tokens are found.
- **Dynamic Reallocation (Resizing):** Within the `while` loop, a check is performed (`if (currentPosition >= capacity)`) to ensure the arguments array does not overflow. If the current capacity is exceeded, the function uses `realloc()` to safely expand the memory

space (`capacity += MAX_ARG_CAPACITY`). This makes the shell robust against commands with an arbitrary number of arguments.

- **Final Structure and Return:** After all tokens have been processed, the essential **NULL terminator** is placed at the end of the array (`arguments[currentPosition] = NULL;`). The execve() function strictly requires this null-terminated arrangement, and the final array of argument pointers is then returned to the main loop for execution.

Path Resolution

The `resolveCommandPath()` function is responsible for locating the full, executable path of a command, a crucial step that allows the user to execute programs without specifying their full directory path.

- **Direct Path Check (Bypass PATH):** Initial verification involves using `strchr()` to determine if the commandName includes a forward slash (`/`). If a slash is present, the function assumes the user provided a full or relative path, bypassing the extensive `PATH` search. It immediately checks for existence and executability using `access(commandName, X_OK)` and returns a duplicate of the path (`strdup()`) if successful, or `NULL` otherwise.
- **Retrieving and Tokenizing `PATH`:** If a direct path is not provided, the function retrieves the system's `PATH` environment variable using `getenv("PATH")`. It then creates a duplicate of this variable (`pathCopy = strdup(pathEnv)`) because `strtok()` modifies the string it operates on, which would otherwise corrupt the original environment variable. The `pathCopy` is tokenized using the colon (`:`) delimiter to isolate individual directories.
- **Iterative Directory Search:** The code enters a `while` loop to iterate through every `directory` found in the tokenized `PATH`.
  - **Path Construction:** Inside the loop, `snprintf()` is used to construct the full potential path (e.g., `/bin/ls`) by concatenating the directory, a forward slash, and the `commandName`.
  - **Access Verification:** The `access(fullPathBuffer, X_OK) == 0` check is performed. If the system confirms that the assembled path leads to an existing file with **execution permission** (`X_OK`), the path is duplicated (`strdup()`) into `foundPath` and the loop terminates (`break`).
- **Cleanup and Return:** Regardless of success or failure, the dynamically allocated memory for the `pathCopy` must be released via `free(pathCopy)`. The function then returns the `foundPath` (which contains the full executable path if found, or `NULL` if the command was not located in any directory listed in the `PATH`).

I/O Stream Management and Multi-Stage Pipelines

- `handleRedirection(char **args)`**:** This helper function is responsible for parsing and applying I/O redirection settings. It is designed to be called **exclusively within the child process** (after `fork` but before `execve`) to ensure that file descriptor modifications do not affect the persistent shell process.

- **File Opening with Specific Flags:** Upon detecting a redirection token, the code invokes `open()` on the subsequent argument (`args[i+1]`). The behavior is strictly controlled by bitwise flags from `<fcntl.h>`:
  - For overwrite (`>`): The flags `O_WRONLY | O_CREAT | O_TRUNC` are used. `O_TRUNC` ensures the file size is reset to zero bytes before writing.
  - For append (`>>`): The flag `O_APPEND` replaces `O_TRUNC`, ensuring new data is added to the end of the file.
  - All creation calls use mode `0644` to set standard read/write permissions.
- **Descriptor Swapping (`dup2`):** The core mechanism uses `dup2(fd, TARGET_FILENO)`. For output redirection, the code executes `dup2(fd, STDOUT_FILENO)`, which closes the default terminal output (descriptor 1) and clones the file's descriptor into that slot. Similarly, `STDIN_FILENO` (descriptor 0) is replaced for input redirection.
- **Argument List Sanitization:** To prevent the executed program from interpreting the redirection syntax as command-line arguments, the code performs `args[i] = NULL`. This effectively truncates the argument vector passed to `execve()`, hiding the `>` token and the filename from the target executable.

- `executePipedCommands(char *inputLine)`: This function orchestrates the execution of multi-stage pipelines, handling the complex task of Inter-Process Communication (IPC).
  - **Pipeline Parsing:** Unlike the standard execution flow, this function first tokenizes the raw input string using the pipe delimiter (`|`) to separate individual commands.
  - **Sequential Forking Loop:** It iterates through the commands, creating a new child process for each stage. For every stage except the last, a communication channel is created using `pipe(pipefd)`.
  - **FD Chaining:** The function maintains logic to link processes:
    1. It connects the **Output** of the current process to the write-end of the current pipe via `dup2`.
    2. It connects the **Input** of the current process to the read-end of the *previous* pipe.
  - **Deadlock Prevention via Reference Counting:** The most critical logic in `executePipedCommands` is the strict management of file descriptors in the parent process. This addresses the behavior of the Unix `read()` system call, which blocks indefinitely until data is available or an **End-Of-File (EOF)** signal is received.
    - **The Kernel Mechanic:** The OS kernel maintains a *reference count* for the write-end of a pipe. An `EOF` is sent to the reading process only when this reference count drops to zero (i.e., all processes holding the write-end have closed it).
    - **The Forking Hazard:** When `fork()` is called, the parent's open file descriptors are duplicated into the child. Consequently, both parent and child hold open handles to the pipe's write-end.
    - **The Solution:** Inside the parent's execution block, the code explicitly executes `close(pipefd[1])`. Without this step, the parent would inadvertently keep the write-end open. The reading child process (the next stage in the pipeline) would never receive an `EOF`, causing it to hang indefinitely waiting for input from a parent that never intends to write, resulting in a deadlock.
  - **Job Control System:** All processes within a pipeline are assigned to the same Process Group ID. The first process created becomes the group leader (setpgid(0, 0)), and all subsequent processes are joined to that same group. By grouping the

processes, the shell ensures that terminal signals like SIGINT (Ctrl+C) or SIGTSTP (Ctrl+Z) affect the entire pipeline simultaneously. If the pipeline is running in the foreground, the shell uses tcsetpgrp() to grant the entire pipeline's PGID control of the terminal. The shell identifies the last_pid of the pipeline and uses it with waitpid(last_pid, &status, WUNTRACED) to determine when the entire chain has completed or been suspended. The entire pipeline is recorded as a single entry in the jobs table, using the command line string and the last_pid for tracking.

Job Control

The Phase 3 leverages advanced POSIX system calls to manage Job Control, enabling the shell to handle multiple concurrent process groups and their interaction with the terminal environment. This priority is critical because commands like cd must execute internally using the chdir() system call to modify the shell's own Current Working Directory; if executed in a child process via fork(), the change would only affect the transient child and leave the parent shell's environment unchanged. This priority logic is expanded to include Job Manipulation commands—jobs, fg, bg, and kill—which directly access the internal job table containing up to MAX_JOBS (20) entries.

- **Job ID:** When a new process is created, add_job performs a search to find the smallest available Job ID (JID) starting from 1. If a process terminates and its ID (e.g., JID 2) is removed from the table by delete_job, that specific ID becomes the first candidate for the next command launched, regardless of how many higher IDs are currently active. he function verifies that the Job ID does not exceed 20; if the table is full, it returns an error to prevent memory overflows. Each job entry maps the user-friendly JID to both the system PID (Process ID) and the PGID (Process Group ID), which is essential for signal isolation.

- **Foreground:** The built-in execution function do_fg() moves a job to the foreground by sending SIGCONT to the entire process group, granting terminal ownership via tcsetpgrp() and blocking the shell using waitpid() with the WUNTRACED flag to monitor termination or suspension. The shell sends a SIGCONT signal to the entire process group (-job-¿pgid). This ensures that if the job was in a JOB_STOPPED state (e.g., via Ctrl+Z), it resumes execution immediately. The shell executes tcsetpgrp(shell_terminal, job− >pgid), which instructs the terminal driver to direct all input and signals (like SIGINT) to the child's group. The parent shell enters a blocking state via waitpid(job-¿pid, &status, WUNTRACED). This prevents the shell from displaying a new prompt until the foreground job either terminates or is suspended again. If the user presses Ctrl+Z, the function detects the stop signal, prints a status message (e.g., [1]+ Stopped), and updates the job's state to JOB_STOPPED. If the process exits normally (WIFEXITED) or is killed by a signal (WIFSIGNALED), the function calls delete_job(). This frees the slot in the MAX_JOBS array, allowing the JID to be recycled for future commands.

- **Background:** The built-in execution function do_bg resumes a stopped job in the background by sending SIGCONT but keeping the shell as the owner of the terminal, allowing the user to continue typing. The function first parses the user input to find the Job ID (JID), which is typically prefixed with a '%' symbol (e.g., bg %1). Then it searches the internal job table—which tracks up to MAX_JOBS (20) entries—to locate the corresponding job_t structure. Once the job is found, the shell sends a SIGCONT signal to the entire process group (-pgid). This "wakes up" all processes in that group, regardless of

whether it is a single command or a pipeline. Crucially, do_bg does not call tcsetpgrp() or waitpid(). This allows the process to run in the background while the shell immediately returns to shellMainLoop to display a new prompt.

- **Kill:** The do_kill function allows for the forced termination of a specific job group. In TinyShell, this is more sophisticated than a simple process kill because it targets Process Groups (PGIDs). Similar to do_bg, the function identifies the target process group using the user-provided JID (e.g., kill %2). Instead of sending a signal to a single PID, do_kill sends the SIGTERM signal to the negative PGID (e.g., kill(-job-¿pgid, SIGTERM)). By targeting the PGID, the shell ensures that if the job is a pipeline (e.g., ls — grep), all processes within that pipeline are terminated simultaneously. The function itself does not remove the job from the table. Instead, it relies on the sigchld_handler to detect the termination signal, reap the zombie process via waitpid(), and call delete_job() to free the slot for ID recycling.

Main Function

The `main()` function serves as the entry point for the entire TinyShell program, adhering to the standard convention of C programs.

- **Signal Setup:** Before entering the execution loop, the function calls setup_signals() to configure the shell's response to terminal interrupts and init_jobs() to clear the internal job table.
- **Terminal Identification:** It identifies the terminal by setting shell_terminal to STDIN_FILENO, ensuring the shell knows which device to control during process group hand-offs.
- **Process Group Leadership:** he shell ensures it is the leader of its own process group by calling getpid() and setpgid(), preventing it from being inadvertently terminated by signals intended for its children.
- **Shell Initialization:** The function's primary responsibility is to initialize and start the shell's core operation by executing the main loop: `shellMainLoop()`. This call transfers control to the continuous command interpreter cycle, where all user interaction and process management occur.
- **Program Termination:** The `shellMainLoop()` function is designed to run until a specific termination condition is met (either `exit` or `EOF`). When `shellMainLoop()` eventually returns control back to `main()` (only upon an `EOF` signal), the `main` function executes `return EXIT_SUCCESS`.
- **Exit Status (Host System):** The `return EXIT_SUCCESS` statement ensures that the TinyShell program itself exits with a status code of **0** to the host operating system (e.g., the WSL Bash environment), signifying that the shell process completed its task successfully.

Compilation and Dependency Management

This section outlines the necessary steps and external dependencies required to successfully compile the TinyShell project, integrating both standard POSIX system calls and the external Readline library.

- **External Library Dependencies (Installation):** Before compilation, the following development packages must be installed on the Linux environment (WSL):
  - `sudo apt install libreadline-dev`: Required for the `<readline/readline.h>` header files.
  - `sudo apt install build-essential`: Required for the core POSIX headers, including `<sys/wait.h>`, and the `gcc` compiler.
  - `sudo apt install trace`: The trace command is required for the system call monitoring. It allows the developer to observe the specific system calls the shell makes, such as fork(), execve(), and waitpid(). It helps verify that signals like SIGCHLD, SIGINT, and SIGTSTP are being sent and received correctly by the process groups. It helps trace the flow of data through pipes and ensures that file descriptors are being closed correctly to prevent deadlocks.
- **The Compilation Command:** The build process utilizes a provided Makefile within the file to correctly link the external readline library. The Makefile manages the dependency chain and ensures the compiler is executed with the correct command-line arguments, including the -lreadline linker flag.The file is compiled using the GNU C Compiler (`gcc`) with specific flags to manage warnings and link external libraries.To compile the program, navigate to the source directory and execute the following command: make.
  Otherwise the full command is:

```
gcc -Wall tinyshell.c -o tinyshell -lreadline
```

  - `-Wall`: Enables all useful compiler warnings, aiding in debugging and code quality.
  - `-o tinyshell`: Specifies the name of the output executable file.
  - `-lreadline`: **Linker Flag.** This critical flag instructs the linker to integrate the compiled code of the `readline` library into the final executable, resolving references for functions like `readline()` and `add_history()`.

Test Analysis and Visual Validation of TinyShell

The following analysis corresponds to the provided screenshot, which demonstrates the successful implementation of all core functionalities required for TinyShell Phase 1 (tinyshell display,process execution, Path Resolution, and exit status reporting).

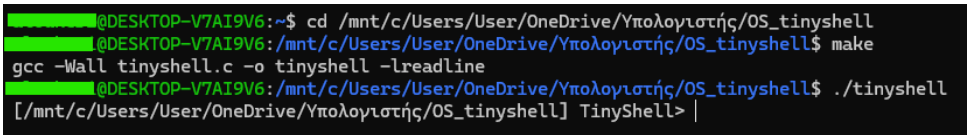


Figure 1: The TinyShell prompt (`TinyShell>`) displayed via the Readline library, ready for input.

Analysis of specific requirements

- `echo Phase 1 of TinyShell Complete!` / `ls -l` **(Exit Status: 0):**

    * **Functionality Verified:** Confirms the correct operation of the entire `fork-execve-waitpid` sequence. Both commands execute successfully and return the expected zero (`0`) exit status, proving that the shell can correctly launch external programs.
- `/bin/pwd` **(Exit Status: 0):**
    * **Functionality Verified:** Confirms that the shell correctly handles commands given via their **full absolute path**. The command executes, prints the path, and returns a successful status of `0`, bypassing the Path Resolution search.
- `true` **(Exit Status: 0)** and `false` **(Exit Status: 1):**
    * **Functionality Verified:** These tests are the definitive proof that the shell accurately reports the child process's termination status. `true` (which always exits successfully) reports `0`, while `false` (which always exits with an error) correctly reports `1`, confirming the proper use of the `WEXITSTATUS` macro.
- `invalid command` **(Error Handling):**
    * **Functionality Verified:** Demonstrates correct failure handling when the executable is not found. The output `myshell: Error: Command not found: invalid` verifies that the `resolveCommandPath` function returned `NULL`, and the shell successfully caught this error, preventing unnecessary process creation.
- `mkdir newdir` **(Exit Status: 0):**
    * **Functionality Verified:** This procedure assesses the fidelity of tokenization and argument transmission. The command `mkdir newdir` confirms that the `splitInputIntoArguments` function accurately segments the input line, ensuring the external binary receives the correct directory name (`newdir`) as a discrete argument in the child process's execution context.

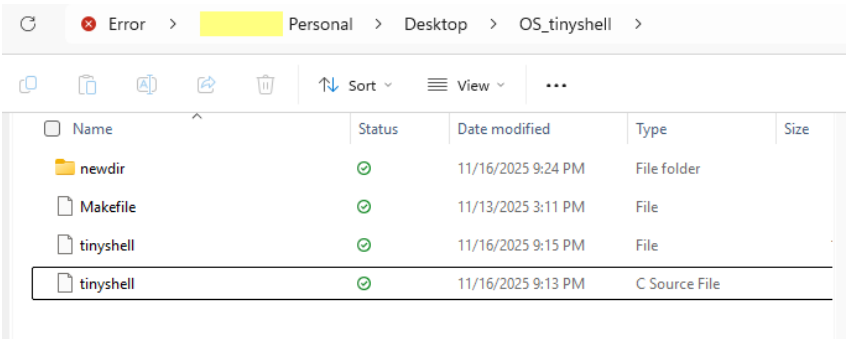
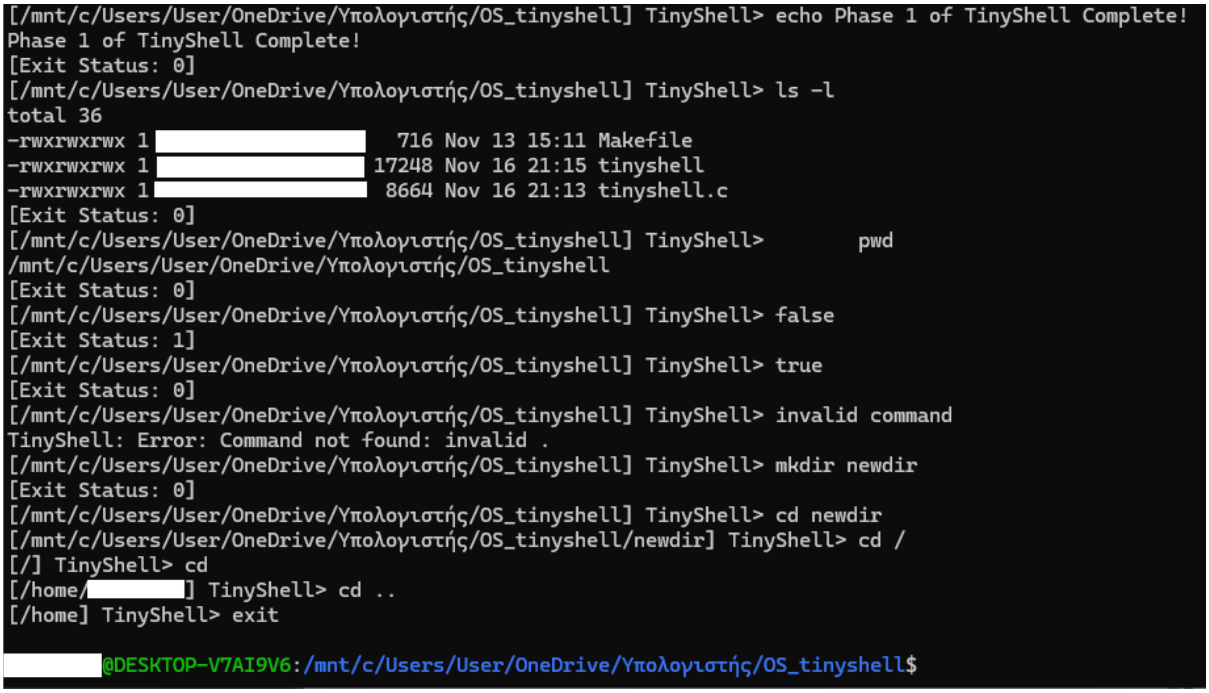


Figure 2: The directory newdir was created successfully.

- `cd newdir`
    * **Functionality Verified**:This core test verifies that the shell identifies `cd` via `strcmp` before attempting a `PATH` search. The invocation of `chdir()` must modify the shell's Current Working Directory (CWD), and this change must be instantly reflected in the prompt string constructed by the `getInputLine` function using `getcwd()`.
- `cd /`
    * **Functionality Verified**:The aim is to check the reliability of the `chdir()` call when presented with an absolute file system path (e.g., the root directory `/`). This test ensures the built-in function operates independently of the shell's current positional context.

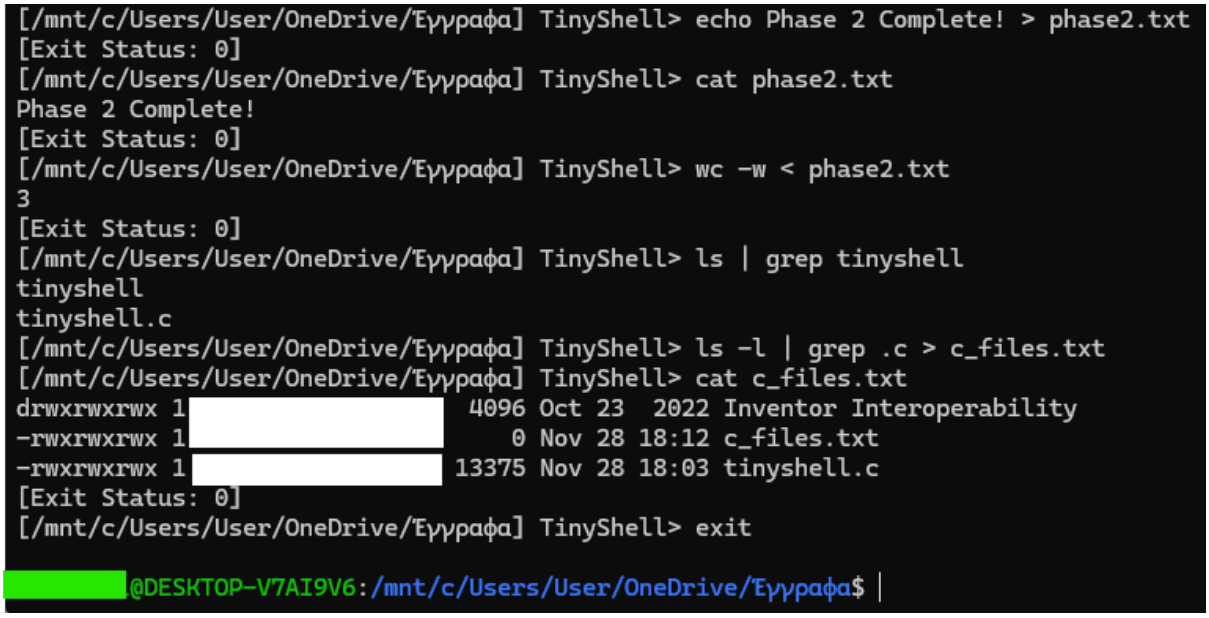


```
[/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell] TinyShell> echo Phase 1 of TinyShell Complete!
Phase 1 of TinyShell Complete!
[Exit Status: 0]
[/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell] TinyShell> ls -l
total 36
-rwxrwxrwx 1 [redacted]  716 Nov 13 15:11 Makefile
-rwxrwxrwx 1 [redacted] 17248 Nov 16 21:15 tinyshell
-rwxrwxrwx 1 [redacted] 8664 Nov 16 21:13 tinyshell.c
[Exit Status: 0]
[/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell] TinyShell>        pwd
/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell
[Exit Status: 0]
[/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell] TinyShell> false
[Exit Status: 1]
[/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell] TinyShell> true
[Exit Status: 0]
[/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell] TinyShell> invalid command
TinyShell: Error: Command not found: invalid .
[/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell] TinyShell> mkdir newdir
[Exit Status: 0]
[/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell] TinyShell> cd newdir
[/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell/newdir] TinyShell> cd /
[/] TinyShell> cd
[/home/[redacted]] TinyShell> cd ..
[/home] TinyShell> exit

[redacted]@DESKTOP-V7AI9V6:/mnt/c/Users/User/OneDrive/Υπολογιστής/OS_tinyshell$
```

Figure 3: Composite validation of TinyShell Phase 1, demonstrating correct execution, Exit Status reporting, and error handling.

- `echo Phase 2 Complete!  > phase2.txt`
  - **Functionality Verified:** This test confirms the correct implementation of the **output redirection operator (>)**. It validates that the `handleRedirection` function successfully opens the target file using the `O_WRONLY | O_CREAT | O_TRUNC` flags. Crucially, it proves that `dup2()` correctly replaced the process's `STDOUT_FILENO`, directing the data stream to the file instead of the terminal.
- `wc -w < phase2.txt`
  - **Functionality Verified:** Verifies the **input redirection operator (<)**. It confirms that the shell opens the specified file in `O_RDONLY` mode and maps it to `STDIN_FILENO` using `dup2()`. The `wc` (word count) command successfully reads the content "3" from the file stream rather than waiting for keyboard input, demonstrating correct file descriptor manipulation.
- `ls | grep tinyshell`
  - **Functionality Verified:** Validates the **Inter-Process Communication (IPC)** mechanism via the pipe operator (`|`). It proves that `executePipedCommands` correctly creates a unidirectional data channel using `pipe()`. The test confirms that the standard output of `ls` was successfully connected to the standard input of `grep`, allowing real-time filtering of the file list without intermediate files.
- `ls -l | grep .c > c_files.txt`
  - **Functionality Verified:** Demonstrates the shell's capability to handle **composite commands** involving both multi-stage pipelines and output redirection simultaneously. This test confirms that the final process in the pipeline (`grep`) correctly inherited the redirected output stream, writing the filtered results (files ending in .c) directly to `c_files.txt` instead of the console.



```
[/mnt/c/Users/User/OneDrive/Έγγραφα] TinyShell> echo Phase 2 Complete! > phase2.txt
[Exit Status: 0]
[/mnt/c/Users/User/OneDrive/Έγγραφα] TinyShell> cat phase2.txt
Phase 2 Complete!
[Exit Status: 0]
[/mnt/c/Users/User/OneDrive/Έγγραφα] TinyShell> wc -w < phase2.txt
3
[Exit Status: 0]
[/mnt/c/Users/User/OneDrive/Έγγραφα] TinyShell> ls | grep tinyshell
tinyshell
tinyshell.c
[/mnt/c/Users/User/OneDrive/Έγγραφα] TinyShell> ls -l | grep .c > c_files.txt
[/mnt/c/Users/User/OneDrive/Έγγραφα] TinyShell> cat c_files.txt
drwxrwxrwx 1 [redacted]  4096 Oct 23  2022 Inventor Interoperability
-rwxrwxrwx 1 [redacted]     0 Nov 28 18:12 c_files.txt
-rwxrwxrwx 1 [redacted] 13375 Nov 28 18:03 tinyshell.c
[Exit Status: 0]
[/mnt/c/Users/User/OneDrive/Έγγραφα] TinyShell> exit

[redacted]@DESKTOP-V7AI9V6:/mnt/c/Users/User/OneDrive/Έγγραφα$
```

Figure 4: This visual validation of Phase 2 demonstrates the successful execution of output redirection (>), input redirection (<), and inter-process communication via pipelines (|).

- `sleep 60 &`
  - **Functionality Verified:** This confirms the correct operation of the background detection logic and asynchronous process creation. The shell identifies the & operator, launches the process in a new process group, and immediately returns the prompt after printing the job notification [1] 614.
- `jobs`
  - **Functionality Verified:** This command validates the shell's ability to maintain and display its internal job table. The output correctly lists the job ID, its current state (Running), and the original command line (sleep 60).
- `Ctrl-Z`
  - **Functionality Verified:** Proves that the shell correctly handles SIGTSTP (Ctrl-Z) to suspend foreground processes. When Ctrl-Z is pressed during sleep 100, the shell intercepts the signal, stops the child process, regains terminal control, and reports the status change as [2]+ Stopped sleep 100.
- `bg %1, fg %1, kill %1`
  - **Functionality Verified:** Confirms the implementation of the built-in bg command and the use of the SIGCONT signal and the shell's capability to transfer terminal control back to a job group using tcsetpgrp(). The shell resumes the previously stopped or backgrounded job, updating its state to Running without blocking the main terminal loop and then with fg it brings the specified job to the foreground and enters a blocking waitpid() until the process terminates or is stopped again. The Job Termination confirms that the shell can send termination signals to specific process groups based on their Job ID. The output Job [1] (628) terminated verifies that the shell successfully dispatched the signal and reaped the process, freeing the JID for recycling.
- `exit` **(Graceful Termination):**

* **Functionality Verified:** Confirms the correct behavior of the built-in `exit` command. The shell performs necessary cleanup and terminates, returning control to the host WSL/Bash shell.

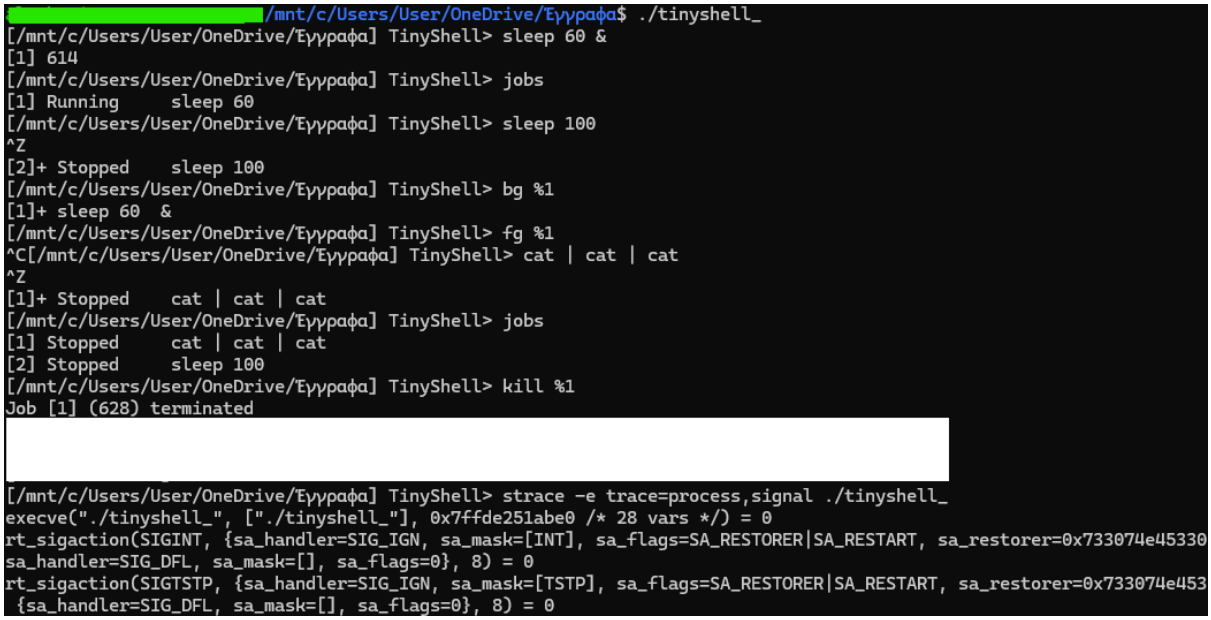


Figure 5: This visual validation of Phase 3 demonstrates the successful execution of job control via foreground fg %1 and background bg %1 management, the ID recycling and the job termination via kill %1.

The successful outcome of all these tests confirms that TinyShell Phase ,2 and 3 fully meet all specifications regarding execution flow, I/O redirection, pipelining, status reporting and job control - process management.

References

1. Modern Operating Systems - Andrew S.Tatenbaum
2. System Calls POSIX in C - Stack Overflow
3. POSIX Library Functions - Kompf.de
4. The GNU Readline Library - GNU.org